

Heterogeneous Robot Collaboration in Unstructured Environments with Grounded Generative Intelligence

Zachary Ravichandran, Fernando Cladera, Ankit Prabhu, Jason Hughes,
Varun Murali, Camillo Taylor, George J. Pappas, Vijay Kumar

Abstract—Heterogeneous robot teams operating in realistic settings often must accomplish complex missions requiring collaboration and adaptation to information acquired online. Because robot teams frequently operate in unstructured environments — uncertain, open-world settings without prior maps — subtasks must be *grounded* in robot capabilities and the physical world. While heterogeneous teams have typically been designed for fixed specifications, generative intelligence opens the possibility of teams that can accomplish a wide range of missions described in natural language. However, current large language model (LLM)-enabled teaming methods typically assume well-structured and known environments, limiting deployment in unstructured environments. We present SPINE-HT, a framework that addresses these limitations by grounding the reasoning abilities of LLMs in the context of a heterogeneous robot team through a three-stage process. Given language specifications describing mission goals and team capabilities, an LLM generates grounded subtasks which are validated for feasibility. Subtasks are then assigned to robots based on capabilities such as traversability or perception and refined given feedback collected during online operation. In simulation experiments with closed-loop perception and control, our framework achieves nearly twice the success rate compared to prior LLM-enabled heterogeneous teaming approaches. In real-world experiments with a Clearpath Jackal, a Clearpath Husky, a Boston Dynamics Spot, and a high-altitude UAV, our method achieves an 87% success rate in missions requiring reasoning about robot capabilities and refining subtasks with online feedback. More information is provided on the project webpage: <https://zacravichandran.github.io/SPINE-HT>.

I. INTRODUCTION

Heterogeneous robot collaboration enables missions such as inspection, logistics, and emergency response, where each robot contributes a unique set of abilities. For example, consider the triaging mission outlined in Fig. 1. An effective planner may first task a UAV to build an overhead map and locate key infrastructure, faster ground vehicles may perform more detailed inspection over UAV-mapped areas, rugged robots may traverse harsh terrain, while others may act as communication relays. In such scenarios, effective collaboration requires reasoning about *subtask dependencies* (e.g., mapping must precede inspection). The planner must also assign subtasks that leverage *unique robot capabilities and constraints*. Finally, because missions occur in unstructured environments— where relevant semantics are unknown,

V. Murali is with the Dept. of Electrical and Computer Engineering at Texas A&M University. All other authors are with the GRASP Laboratory at the University of Pennsylvania. Correspondence can be addressed to zacravi@seas.upenn.edu. We acknowledge support from ARL DCIST CRA W911NF-17-2-0181, NSF Grant CCR-2112665, and the NSF Graduate Research Fellowship Program.

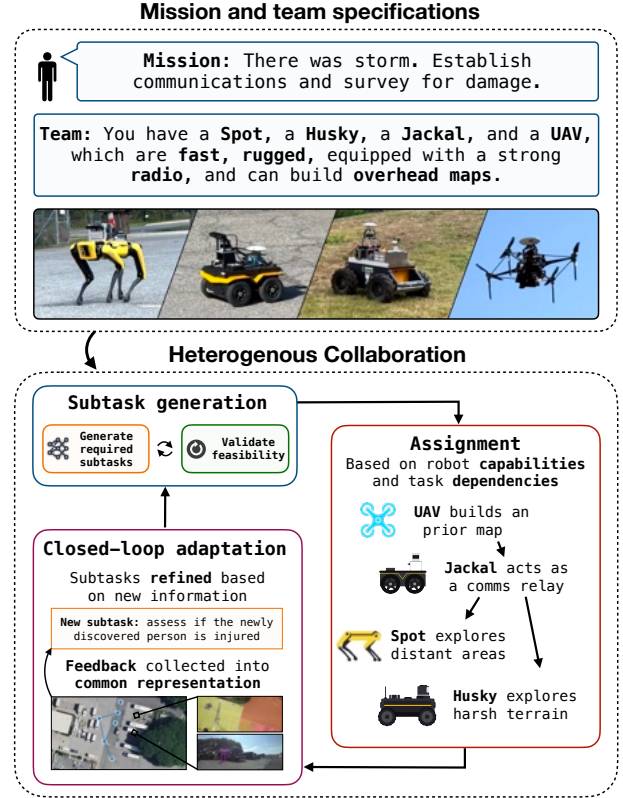


Fig. 1. SPINE-HT takes as input mission and team specifications in natural language. SPINE-HT then generates grounded subtasks that are validated for realizability while preserving dependencies, assigns subtasks based on robot capabilities, then refines subtasks given robot feedback acquired during online operation.

prior maps are unavailable, and robots encounter noise in sensing and actuation—the team must *refine subtasks based on feedback acquired online*, such as map updates or subtask results. While the research community has advanced this vision, achieving such capable robot teams remains an elusive challenge with current approaches, which have been developed for structured environments (*i.e.*, simulation or controlled lab settings), where mission goals are well-specified, robot affordances are noiseless, and the world model is known or simple [1], [2], [3], [4], [5], [6], [7].

Multi-robot teaming has traditionally been addressed via optimization-based methods, where goals are encoded in objective functions and team capabilities are described via constraints [5], [6]. Optimization-based approaches have addressed large-scale collaboration problems such as distributed mapping, and they offer a principled approach to

task allocation. However, these methods typically assume complete mission specifications, a detailed team description, and *a priori* knowledge of the environment, limiting their applicability in large-scale and unstructured settings.

A second line of multi-robot teaming research focuses on deployment in unstructured environments [8], [9], [10], [11]. The success of these systems largely comes from an emphasis on *closed-loop* operation — the planner is not considered in isolation, rather it is deployed in-the-loop with perception and inter-robot communication. And while these approaches improve robustness, they continue to rely on predefined semantics, detailed team specifications, and fixed mission goals, and thus cannot accommodate incomplete or evolving mission specifications.

The increasing maturity of generative intelligence — namely large language models (LLMs) and vision language models (VLMs) — promises to enable multi-robot teaming approaches that are capable of understanding language-specified missions and reasoning about appropriate subtasks. Yet, leveraging these models for heterogeneous teaming in unstructured environments remains a significant challenge. While several LLM-enabled approaches have demonstrated multi-robot teaming given language specifications, existing work has primarily been designed for structured environments such as simulators or laboratories [1], [2], [3], [4].

To bridge the gap between generative intelligence and robot collaboration in unstructured environments, we propose SPINE-HT, which extends the SPINE [12] framework for heterogeneous teaming (HT) by grounding LLM-enabled reasoning in the physical and semantic context of a teams via closed-loop plan validation and semantic mapping. As illustrated in Fig. 2, SPINE-HT takes as input a mission and team specifications in natural language, then uses an LLM to generate *grounded* and validated subtasks — subtasks with mission-relevant semantics that respect the physical constraints of both the robot and environment. This generation process leverages a directed acyclic graphical (DAG) structure to identify dependencies while allowing for parallel execution of independent subtasks. Subtasks are assigned to robots based on capabilities within the team. During task execution, robots collect feedback comprising semantic map updates and subtask outcomes, and this feedback is leveraged by our framework to refine subtasks until the mission is complete. To summarize, our **contributions** are a framework that, given language specifications:

1. Generates semantically and physically grounded subtasks that model inter-robot dependencies.
2. Assigns subtasks based on robot capabilities within a heterogeneous team.
3. Refines subtasks online based on feedback collected from mapping and plan validation, enabling operating in unstructured environments.

We evaluate these contributions via simulated and real-world experiments with closed-loop mapping and control. In real-world experiments, our framework achieves a mission success rate of 87%, while producing plans that are within 15% efficient in terms of subtasks required. Compared to

state-of-the-art LLM-enabled heterogeneous planning approaches, our method achieves nearly twice the mission success rate and subtask efficiency.

II. RELATED WORK

Heterogeneous robot teaming has been addressed via optimization and formal methods, robust systems for deployment in unstructured environments, and most recently LLM-enabled methods — *i.e.*, generative intelligence.

Optimization and formal methods. Heterogeneous robot planning has traditionally been formulated as an optimization problem, where tasks are encoded via objective functions and constraints govern which robot may be assigned to a given task [5]. For example, a coverage control problem may yield an objective that specifies robot locations, and constraints may encode robot capabilities such as its sensing modalities or speed, and this formulation admits tractable solutions via constrained optimization approaches [6]. By incorporating further information such as robot dynamics, optimization approaches may be employed to jointly solve task allocation and motion planning [7], [13]. [14] employs graph-based representations to model inter-robot task dependencies, which enables multi-robot allocation over complex tasks. Formal methods specifications, such as linear temporal logic (LTL) or Planning Domain Definition Language (PDDL), have also been used to represent multi-robot missions [15] and provide guarantees on correctness and feasibility. These approaches require an expert user to explicitly encode robot capabilities, detailed mission specifications including task dependencies, and assume a known world model.

Multi-robot systems in unstructured environments. While the above methods typically consider well-structured environments, a line of work develops multi-robot teaming approaches for unstructured environments, where the world is unknown, the terrain is hazardous, and the robots may not enjoy reliable communication [16], [17], [18]. These systems have typically been designed for missions requiring exploration and object search, with teams comprising air and ground robots [9], [10], [8]. The success of these systems largely relies on planners embedded in robust feedback loops with perception, control, inter-robot communication, and other key modules [19], [20], [11]. While the ability of these systems to operate in large-scale and unstructured environments is impressive, these systems are designed for *fixed mission and team specifications*. Adapting to new specifications requires development by an expert user.

Generative intelligence for robot teams. Recent work leverages the generative intelligence of LLMs to translate linguistic mission and robot specifications for multi-robot collaboration. Some approaches focus on homogeneous robot teams in centralized or decentralized manners [2], [21] incorporate feedback from robot task execution (*i.e.*, success or failure) for replanning, but do not consider closed-loop perception. SMART-LLM [1] and similar works consider heterogeneous robot collaboration [4], [22]. The proposed methods use an LLM to decompose language specifications

SPINE-HT Framework

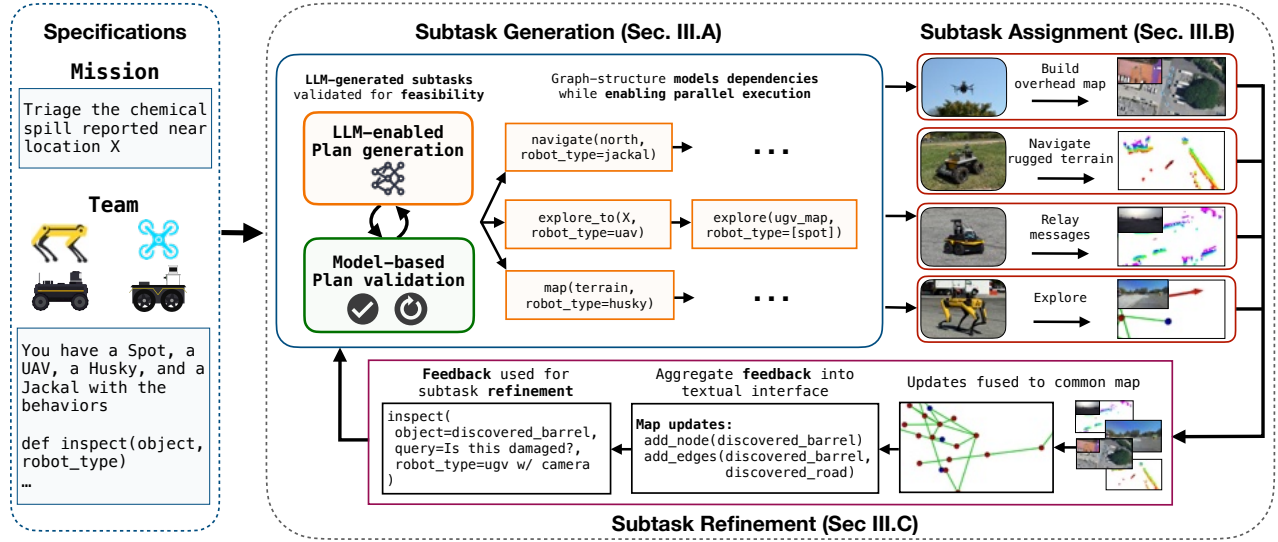


Fig. 2. SPINE-HT takes as input natural language specifying mission goals and team capabilities. Our framework then uses an LLM to generate grounded subtasks with their dependency ordering (§III-A). Subtasks are then assigned to robots based on capability (§III-B). Feedback comprising semantic map updates outcomes are collected and used for subtask adaptation (§III-C).

into robot-specific tasks, but they require fully-known world models. COHERENT furthers LLM-enabled heterogeneous robot collaboration by incorporating feedback mechanisms for plan validation and updated world models [3]. However, this method does not model task dependencies, assumes simplistic world models, and its feedback mechanisms rely on auxiliary LLMs that are also prone to hallucination. [23] uses an LLM to translate language commands into a PDDL plan in a grounded world model. However, their method requires a fully-known map and is limited to coordination with fully-specified goals (*i.e.*, a user gives specific instructions to each robot). [24] recently proposed a method for air-ground robot teaming with language-specified missions, but their method is not scalable to different robot teams. In summary, existing LLM-enabled heterogeneous robot teaming methods lack robust mechanisms for *grounding* semantic goals in robot capabilities and physical constraints, preventing operation in unstructured environments.

III. SPINE-HT

SPINE-HT addresses heterogeneous robot collaboration through a three-stage framework that maps natural language mission specifications into executable, grounded subtasks that are iteratively refined throughout the mission. As illustrated in Fig. 2, our method comprises three core modules that operate in a closed-loop fashion. First, the *subtask generation* module maps (possibly incomplete) language specifications into a directed acyclic graph (DAG) of *grounded subtasks* — subtasks with metric and semantic objectives that respect the physical constraints and capabilities of both the robot and its environment. The DAG structure explicitly models inter-task dependencies, enabling parallel execution where possible while maintaining logical ordering constraints (§III-A). Next, the *subtask assignment* module allocates tasks to robots based on their capability, considering both hard

constraints (*e.g.*, physical feasibility) and semantic capabilities stated in the language specification (§III-B). Finally, the feedback module incorporates subtask execution results — namely semantic map updates and task outcomes — to refine semantic and physical subtask goals. This feedback enables adaptive operation in unstructured environments where the robot’s knowledge of the environment is constantly evolving.

A. Subtask Generation

The subtask generation module maps natural language specifications into grounded subtasks via LLM-guided reasoning and formal verification. This process comprises three key stages — formulating input specifications, generating grounded subtasks, and subtask validation.

Input specifications. The subtask generation module is provided with incomplete mission and team specifications. Mission specifications simply state the high-level goal (*e.g.*, “triage the area for damage”), while team specifications list the available robots and their collective *behaviors* — high-level functionality that is used to compose plans. Each behavior takes two types of arguments: *grounding argument* — metric and semantic goals — and *capability arguments* — robot ability required for that behavior. For example, the `ground_robot_inspect` behavior:

```
def ground_robot_inspect(object: str, query:
    ↳ str, robot_type: Literal["jackal",
    ↳ "spot"]):
    """Inspect `object` for information described
    ↳ in `query`. Robots "jackal" and "spot" are
    ↳ equipped with RGB-D cameras. """
```

defines the grounding arguments `object` (target entity) and `query` (semantic information to gather). The capability argument `robot_type` constrains execution of that behavior to a robot equipped with an RGB-D camera perform

Behavior	Description	Preconditions
set_labels	set semantic labels	valid semantics
map_region	augment semantic map	region exists
inspect_object	inspect object	object visible
navigate	go to a known location	path exists

TABLE I
SUBSET OF BEHAVIORS CONSIDERED IN OUR WORK.

inspection. Capability arguments may be both robot platform or robot type specific robots, enabling soft capability specifications (*i.e.*, “spot has the best camera in the team”).

Generating mission abstractions. Given the input specifications, the subtask generation module uses an LLM to produce grounded subtasks structured as a directed acyclic graph (DAG), where nodes consist of grounded subtasks (behaviors instantiated with valid arguments) and edges denote inter-task dependencies (see Fig. 2). The DAG structure identifies independent subtasks while respecting dependency constraints — tasks within connected components must run sequentially, while independent components may run concurrently. To improve plan quality and interpretability, the LLM employs chain-of-thought reasoning (CoT), explicitly justifying key steps in its subtasks generation process [25].

Plan validation. While LLMs enable powerful contextual reasoning, their potential to hallucinate information may lead to infeasible plans [12]. LLM-generated plans are therefore verified through an Assume-Guarantee framework before being assigned to the robot team [26]. In this framework, each behavior defines preconditions that must hold for invocation. The validator checks these preconditions against the current world state and provides specific feedback where necessary. For example, the `ground_robot_inspect` behavior discussed above requires that the target `object` is visible and reachable in the current map. If this behavior is invoked with an unreachable object, the LLM would receive the feedback “`object` is unreachable, and you may need to find a path.” This iterative generation-validation feedback loop runs until all subtasks satisfy their defined preconditions, ensuring feasibility before subtasks are assigned to robots. Tab. I summarizes key behaviors available to the planner along with their primary preconditions. Please refer to §A2-B for a complete list of behaviors considered in this work.

B. Subtask Assignment

The task assignment module assigns validated subtasks from the mission graph to specific robots through constrained optimization, accounting for heterogeneous capabilities and minimizing execution costs. At each planning iteration, given the set of generated subtasks $\mathcal{T}$, the module constructs a candidate subtask $\mathcal{T}' \subseteq \mathcal{T}$ comprising tasks ready for assignment, *i.e.*, dependency-free nodes.

The assignment problem is formulated as a linear optimization over the binary decision variable $X \in \{0, 1\}^{R \times |\mathcal{T}'|}$, where R is the number of robots and $X_{rt} = 1$ indicates that robot r is assigned task t . The cost matrix $C \in \mathbb{R}^{R \times |\mathcal{T}'|}$

encodes capability constraints and additional tasking preferences — C_{rt} is large if robot n is not capable of completing task t ; otherwise, the cost is defined as the distance to task or other relevant metrics. This yields:

$$\min_X \sum_{r=1}^R \sum_{t=1}^{|\mathcal{T}'|} C_{rt} X_{rt} \quad (1)$$

$$\text{s.t.} \quad \sum_{t=1}^{|\mathcal{T}'|} X_{rt} = 1 \quad \forall r \in \{1, \dots, R\} \quad (2)$$

$$\sum_{r=1}^R X_{rt} \leq 1 \quad \forall t \in \{1, \dots, |\mathcal{T}'|\} \quad (3)$$

The first constraint ensures that all robots are assigned a task, while the second ensures that tasks are assigned to at most one robot. To handle cases where $|\mathcal{T}'| < R$, we introduce “idle” tasks with costs such that, if a robot cannot achieve any candidate task, it will be assigned as “idle.” Conversely, unassigned tasks are kept as candidates for the next assignment iteration. The assignment process operates iteratively as the mission progresses. Upon completion, subtasks are marked as complete in the DAG, thereby creating an updated active subgraph where previously blocked tasks may be available for assignment. A new candidate subtask set $\mathcal{T}'$ is computed and assigned to the robots, and this process iterates until all tasks in the mission graph have been assigned.

C. Subtask Refinement

Robots continuously gather information about the world during mission execution. The feedback module collects this information, synthesizes it into a structured update, and provides it to the subtask generation module (§III-A) for mission refinement and adaptation. This closed-loop operation is essential for robust operation in unstructured environments, where environmental knowledge is incomplete and constantly evolving. We consider two primary sources of feedback: semantic map updates and subtask outcomes. We first describe the semantic map representation used by our planner and then describe feedback representations.

Online open-set semantic mapping. Our approach models the world via a topological graph-based semantic maps — a rich representation that has facilitated a variety of semantic planning methodologies [12], [24], [23]. The map consists of two node types: *regions* and *objects*. Region nodes indicate traversable locations in freespace, while object nodes correspond to semantic entities in the environment. Map connectivity is defined between two edge types: region-to-region edges define traversable robot paths, while object-to-region edges denote visibility relationship between an object and point in freespace. Importantly, our mapping framework enables *open-set semantics* via open-set object detection and vision-language models (VLMs), which may enrich nodes with open-ended descriptions (*e.g.*, “this car has a dent on its side”). In contrast to planning methods that assume fixed semantics at planning time, our framework must infer and generate mission-relevant semantics [23], [8], [5].

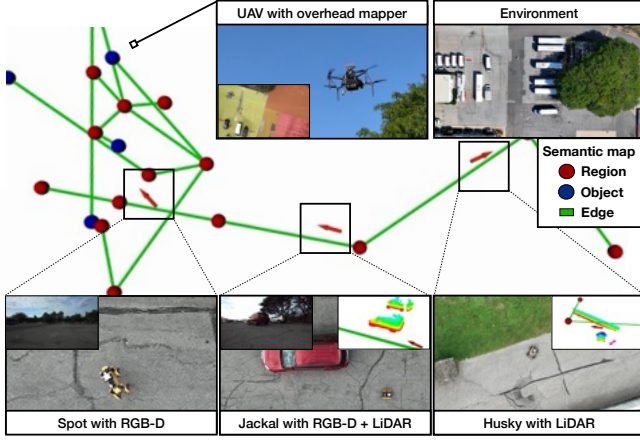


Fig. 3. Our framework aggregates updates from heterogeneous robots into a common semantic map used for subtask refinement.

While our method accepts a prior semantic map (*e.g.*, from aerial imagery or a UAV), each robot constructs a local map online. The sparse, graph-based nature of these semantic maps facilitate online aggregation across varying sensor modalities during operation, where map updates comprise atomic node and edge additions and deletions¹ (see Fig. 3).

Mission feedback and history. The feedback module translates map updates and task outcomes into a textual interface that may be ingested by the subtask generation LLM. Map updates are provided to the LLM via a functional interface that defines core graph operations, such as node and edge additions and removals, as well as more nuanced semantic description. For example, a new `boat` node at location `[10, 20]` connected to a `dock` would yield the update:

```
add_node(name='boat', location=[10, 20],
description='in good condition'
edges=['dock'])
```

Task outcomes are provided via a similar interface that provides key information in a standardized format. For example, a failed navigation subtask may provide the update “Subtask X failed. path between `node_1` and `node_2` was blocked.” All updates at a given planning iteration are provided to the subtask generation module via in-context updates. While storing history via in-context updates leads to accumulating token length, we find that the maximum tokens used in our experiments is roughly 60k, while current LLMs have a context length of over 1M (see §IV-B).

IV. EXPERIMENTS

We conduct experiments to assess SPINE-HT’s ability to fulfill the contributions stated in §1. Namely, the ability to:

- C1.** Infer grounded subtasks that capture dependencies.
- C2.** Assign subtasks based on robot capabilities.
- C3.** Adapt subtasks given feedback acquired online.

¹While we assume consistent localization across robots, future work may incorporate advanced distributed mapping techniques that account for mapping conflicts [10], [27]



Fig. 4. Evaluation environment: real semi-urban office park (top) and three urban, semi-urban, and rural simulation environments (bottom).

To evaluate our first contribution, we evaluate against missions with *incomplete specifications* that require the planner to infer relevant subtasks and their dependencies. To evaluate our second contribution, we design missions requiring heterogeneous collaboration where mission success requires leveraging unique robot capabilities. Finally, to assess our third contribution, we evaluate in environments where the planner’s semantic map is incomplete, requiring the planner to actively explore mission-relevant regions and adapt its plan based on findings acquired online. We conduct extensive evaluation across both simulation and real-world experiments, comprising over 40 distinct missions with up to four heterogeneous robot platforms. Unlike prior LLM-enabled heterogeneous planning work, we conduct all experiments with closed-loop mapping and control [3], [1], [4]. This integration stresses the planner’s ability to interpret and react to realistic sensor inputs, environmental uncertainties, and execution failures — all of which are critical for robust deployment in unstructured environments.

A. Experimental Setup

We now provide an overview of the key experimental components: robotic platforms, dimensions of heterogeneity, evaluation environments, baseline methods, mission specifications, performance metrics, and implementation details.

Experimental platforms. We evaluate with a total of four platforms (see Fig. 1). *Clearpath Jackal*: A small mobile robot equipped with a LiDAR and RGB-D camera. *Clearpath Husky*: A large, rugged mobile platform capable of traversing obstacles including steep slopes and curbs. It is equipped with LiDAR only, limiting semantic perception. *Boston Dynamics Spot*: Quadrupedal robot with superior mobility compared to wheeled platforms. The Spot is equipped with RGB-D camera for semantic perception but lacks LiDAR for detailed metric mapping. *High-altitude UAV*: A quadrotor capable of quickly flying at high altitudes (30 meters) but is limited to overhead sensing.

Dimensions of heterogeneity. Our platforms differ across

Method	Subtask Generation				Capability Reasoning				Exploration			
	Suc. (↑)	ST (↓)	Opt. (↑)	Tok. (↓)	Suc. (↑)	ST(↓)	Opt.(↑)	Tok. (↓)	Suc. (↑)	ST (↓)	Opt. (↑)	Tok. (↓)
Expert	100.0%	1.0	100.0%	-	100.0%	1.0	100.0%	-	100.0%	1.0	100%	-
LLM baseline	37.5%	6.2	81.0%	11.7k	37.5%	5.5	87.0%	10.2k	50.0%	2.5	66.0%	8.7k
Ours	100.0%	1.0	90.0%	3.5k	75.0%	1.3	84.0%	5.1k	100.0%	1.2	86.0%	5.9k

TABLE II
SIMULATION RESULTS ACROSS 24 SPECIFICATIONS.

four key capability dimensions. *Perception*: Sensor modalities dictate platform perception capabilities. In our experiments, RGB-D cameras were used for semantic mapping while LiDAR platforms performed metric mapping. *Traversability*: Platforms vary in the difficulty of terrain they can traverse and speed at which they move. *Communication*: Certain robots carry more powerful radios and may support communication within the team. *Payload*: Certain robots may be equipped with payloads that must be used for mission success. We vary these dimensions to produce *team configurations*, where robots are given individual capabilities or constraints. For example, a robot may have a powerful perception system or a robot may be carrying a package.

Baselines. We compare against the following baselines. First, we consider an Expert Planner, where a user provides explicit mission specifications comprising step-by-step instructions and optimal robot assignments. This baseline provides an upper bound estimate on mission performance and emulates existing approaches that require known world models and explicit mission instructions. We then consider COHERENT, a state-of-the-art LLM-enabled planner for heterogeneous teaming [3]. COHERENT represents the current LLM-based heterogeneous robot planning paradigm, sharing core characteristics with related methods [1], [4], [22] including (1) text-based plans representation (2) no formal verification, (3) no structured feedback mechanisms to track updates and history throughout a mission. To ensure fair comparison, we provide identical behaviors across all methods.

Environments. We evaluate in both simulation and real-world environments as illustrated in Fig. 4. We use a photo-realistic Unity-based simulator featuring large-scale outdoor environments with semantics including factories, offices, communication towers, bridges, and vehicles. We also conduct experiments in an uncontrolled real-world urban office park with semantics such as vehicles, people, vegetation, and infrastructure. Crucially, these environments remained active during experiments, forcing the planner to handle unexpected perception inputs and avoid environment hazards.

Mission Specifications. We design missions to emulate real-world scenarios including infrastructure inspection, emergency response, personal assistance, and environmental monitoring, and we design specifications that assess our three contributions stated in §1. First, *subtask generation*: the planner has to infer generate grounded subtasks from incomplete mission specifications. Second, *capability reasoning*: optimal solutions require leveraging unique robot capabilities. For example, assigning strong-perception platforms to detailed

inspection tasks, while using mobile platforms for wide area coverage. Finally, *exploration*: the planner operates in partially-known environments that require actively exploring mission-relevant areas. Mission success requires the planner to configure robots in a certain location (*e.g.*, for package delivery or communication) and acquire mission-relevant information (*e.g.*, for infrastructure inspection).

Metrics. We report four metrics. *Success rate*: the percentage of missions achieving all required objectives (Suc.). *Subtasks*: number of subtasks required for mission completion relative to expert baseline, for successful missions (ST). *Optimal subtasks assignment*: percentage of subtasks correctly assigned to each robot, as determined by the unique robot capabilities and mission requirements (Opt.). ST and Opt. are reported relative to the expert planner. Finally, we report *tokens* required per subtask generation (Tok.).

Implementation details. We implement our planner using GPT-4.1 as the subtask generation LLM, and we solve the optimization problem described in §III-B via linear programming. Our base semantic mapping framework takes three sources of input: odometry, a Lidar pointcloud, and RGB-D imagery. Odometry and Lidar pointclouds are used to construct region nodes and edges, while an open-vocabulary object detector and VLM are used to identify objects and add mission-relevant descriptions to the map [28], [29]. The UAV performs high-altitude map construction using the framework proposed in [24]. Our framework assumes intermittent communication [20], which requires only the basestation—responsible for running subtask generation and allocation (§III-A, §III-B)—to maintain internet connectivity. Other robots may drop in and out of communication range during the mission. We provide more details in §A2.

B. Simulation Results

We run a total of 24 missions requiring at least 3 to 7 subtasks. We report simulation results across the three mission categories — subtask generation, capability reasoning, and exploration — in Tab. II. Our method achieves an average success rate of 100.0%, 75.0%, and 100.0% across these three mission categories respectively, while the LLM baseline achieves 37.5%, 37.5%, and 50.0%. The LLM baseline also required more subtasks to complete missions (between 2.8 to 6.1 times) and used more tokens (up to 3.3 times). While the subtask optimality of the LLM baseline was comparable to our method for the first two mission categories, it was nearly 20% lower for the exploration missions.

We find that while the LLM baseline produced reasonable semantic goals, it suffered from two primary failure modes.

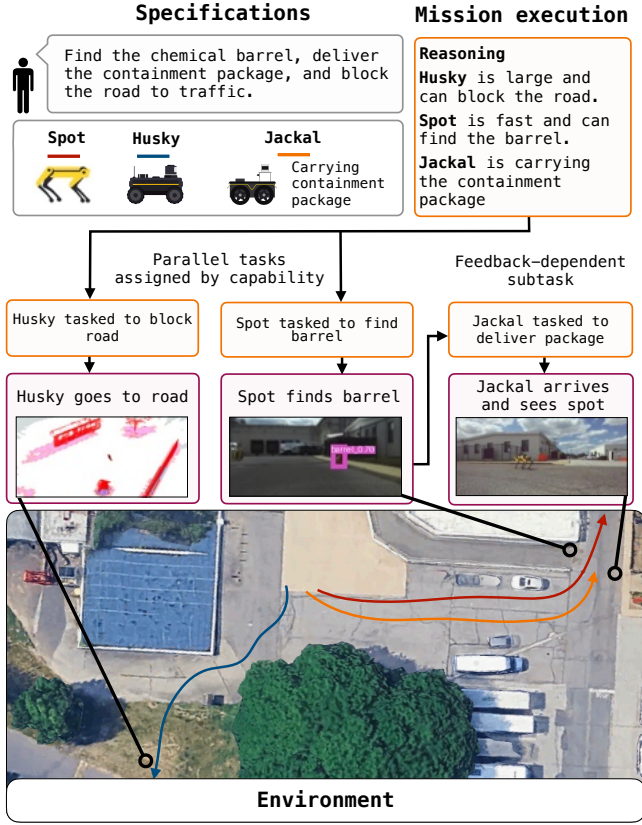


Fig. 5. Example real-world result where the planner must infer grounded subtasks, reason about robot capabilities, and refine subtasks online based.

First, the baseline struggled to efficiently ground plans in the environment, often composing repetitive and suboptimal navigation-related subtasks. These unnecessarily verbose subtasks would increase planning horizon, and often lead to hallucination. Interestingly, this problem was alleviated during exploration missions where the prior map received by the planner was significantly smaller and the LLM baseline was able to leverage the available behaviors to explore mission-relevant locations. In contrast, our method generally produced targeted subtasks that identified key locations in the environment across all three mission scenarios. The LLM baseline also struggled to correctly invoke perception and other robot capabilities. For example, the LLM baseline often did not properly interpret the results of a VLM query, or it would attempt to invoke queries on robots without VLMs.

C. Real-world Results

We ran a total of 16 real-world experiments requiring at least 4 and up to 9 subtasks with teams between 2 to 4 robots, and we report results in Tab. III. Our method achieved a mission success rate of 87.5%, while requiring 1.15 subtasks compared to the expert with a subtask optimality rate of 81%. We observed two primary failure modes. First, we observed failure during a mission requiring exploration. The subtask generator’s LLM attempted to navigate to an infeasible point, and while it received feedback that its plan was invalid, it could not formulate a proper solution within the allowed

Method	Success (↑)	ST (↓)	Opt. (↑)	Tokens (↓)
Expert Planner	1.0	1.0	1.0	-
Ours	87.5%	1.15	81.0%	4.7k

TABLE III

REAL-WORLD RESULTS ACROSS 16 SPECIFICATIONS.

iterations. We also observed instances where our planner did not interpret subtle feedback. In one instance, our method tasked a robot to navigate to a mission-relevant location, however the robot was unable to fulfill that task. The planner did not receive any update from that robot, but it assumed the subtask was successful and terminated the mission.

Apart from these cases, we found that our method was able to ground subtasks, reason about robot capabilities, and adapt subtasks based on information acquired online. We provide an example three robot mission in Fig. 5, where the planner is tasked with triaging a chemical spill. The planner correctly tasks the Husky and Spot in parallel. Once the chemical container is localized and added to the map, the planner tasks the Jackal, which is carrying a chemical containment package. Fig. 2 illustrates a successful four robot mission where the planner is tasked with triaging damage. The planner tasks the UAV to build an overhead map, while the Spot performs longer range exploration, the Jackal serves as a communication relay, and the Husky traverses rugged terrain. Once relevant semantics are found (*i.e.*, a chemical barrel), the planner tasks inspection for detailed analysis.

D. Ablation Studies

Finally, we ablate key components of SPINE-HT on a subset of 6 specifications. We first remove the task allocation procedure described in §III-B, instead having the LLM directly assign robots to subtasks. Next, the subtask generation LLM directly predicts subtasks, instead of using the generation process described in §III-A. Finally, we remove the feedback loop described in §III-C, which requires the LLM to generate open-loop plans.

We report results in Tab. IV. Without the task allocation module, subtask optimality drops from 90.2% to 80.0%. We find this is due to poor spatial reasoning by the LLM — it does not account for distance to goal. Without reasoning for subtask generation, we observe a nearly 17% drop in mission success with 21% less optimal subtasks, indicating that structured generation is crucial for maximally successful and optimal planning. Perhaps unsurprisingly, the planner cannot achieve any missions without feedback, underscoring the importance of online adaptation.

Method	Success (↑)	ST (↓)	Opt. (↑)	Tokens (↓)
Ours	100.0%	1.1	90.2%	3.7k
- Assignment	100.0%	1.2	80.0%	4.5k
- Reasoning	83.3%	1.3	69.0%	2.5k
- Feedback	0.0%	-	-	14.4k

TABLE IV

ABLATION RESULTS AVERAGED OVER 6 SPECIFICATIONS.

V. CONCLUSION

We present SPINE-HT, a heterogeneous robot collaboration framework that grounds the generative reasoning of LLMs into robot capabilities and constraints via a three-phase framework that leverages online plan validation and feedback. Through simulation and real-world experiments comprising air and ground platforms, we demonstrate our framework’s ability to infer grounded subtasks from natural language specifications, assign subtasks based on robot capabilities, and leverage feedback to refine subtasks online. To our knowledge, this is the first framework to demonstrate LLM-enabled heterogeneous teaming in real-world unstructured environments with four distinct platforms.

We envision three promising directions for future work. First, our generation and assignment modules run centrally, and designing decentralized alternatives is a natural extension of our framework. Developing on-device language models for robot teaming, such as through the method proposed in [30], is a promising means of removing reliance on cloud-hosted LLMs and furthering the robustness of our system. Finally, we would also like to consider settings requiring manipulation or other robot modalities.

REFERENCES

- [1] S. S. Kannan, V. L. Venkatesh, and B.-C. Min, “Smart-llm: Smart multi-agent robot task planning using large language models,” *International Conference on Intelligent Robots and Systems*, 2024.
- [2] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, “Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?” *ICRA*, 2024.
- [3] K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao, “Coherent: Collaboration of heterogeneous multi-robot system with large language models,” *ICRA*, 2025.
- [4] Y. Zhu, J. Chen, X. Zhang, M. Guo, and Z. Li, “Dexter-llm: Dynamic and explainable coordination of multi-robot systems in unknown environments via large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.14387>
- [5] A. Prorok, M. A. Hsieh, and V. Kumar, “The impact of diversity on optimal control policies for heterogeneous robot swarms,” *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 346–358, 2017.
- [6] A. e. a. Prorok, “Formalizing the impact of diversity on performance in a heterogeneous swarm of robots,” *ICRA*, 2016.
- [7] A. Messing, G. Neville, S. Chernova, S. Hutchinson, and H. Ravichandran, “Grstaps: Graphically recursive simultaneous task allocation, planning, and scheduling,” *The International Journal of Robotics Research*, vol. 41, no. 2, pp. 232–256, 2022.
- [8] A. Agha-Mohammadi, K. Otsu, B. Morrell, D. D. Fan, R. Thakker, A. Santamaria-Navarro, S.-K. Kim, A. Bouman, X. Lei, J. Edlund, M. F. Ginting, K. Ebadi, and J. ... Burdick, “Nebula: Team costar’s robotic autonomy solution that won phase ii of darpa subterranean challenge,” *Journal of Field Robotics*, 2022.
- [9] I. D. Miller, F. Cladera, A. Cowley, S. S. Shivakumar, E. H. Lee, L. Jarin-Lipschitz, A. Bhat, N. Rodrigues, A. Zhou, A. Cohen, A. Kulkarni, J. Laney, C. J. Taylor, and V. Kumar, “Mine tunnel exploration using multiple quadrupedal robots,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2840–2847, 2020.
- [10] I. D. Miller, F. Cladera, T. Smith, C. J. Taylor, and V. Kumar, “Air-ground collaboration with spomp: Semantic panoramic online mapping and planning,” *IEEE Transactions on Field Robotics*, 2024.
- [11] Y. Chang, K. Ebadi, C. E. Denniston, M. F. Ginting, A. Rosinol, A. Reinke, M. Palieri, J. Shi, A. Chatterjee, B. Morrell, A.-a. Agha-Mohammadi, and L. Carlone, “Lamp 2.0: A robust multi-robot slam system for operation in challenging large-scale underground environments,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, 2022.
- [12] Z. Ravichandran, V. Murali, M. Tzes, G. J. Pappas, and V. Kumar, “Spine: Online semantic planning for missions with incomplete natural language specifications in unstructured environments,” *International Conference on Robotics and Automation (ICRA)*, 2025.
- [13] V. Edwards and M. A. Hsieh, “Distributed adaptive macroscopic ensemble task allocation of heterogeneous robot teams in dynamic environments,” in *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, 2025.
- [14] W. Gosrich, S. Mayya, S. Narayan, M. Malencia, S. Agarwal, and V. Kumar, “Multi-robot coordination and cooperation with task precedence relationships,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5800–5806.
- [15] S. Kalluraya, G. J. Pappas, and Y. Kantaros, “Multi-robot Mission Planning in Dynamic Semantic Environments,” in *IEEE International Conference on Robotics and Automation (ICRA)*, May–June 2023.
- [16] F. Cladera, I. D. Miller, Z. Ravichandran, V. Murali, J. Hughes, M. A. Hsieh, J. Taylor, C. and V. Kumar, “Challenges and opportunities for large-scale exploration with air-ground teams using semantics,” *arXiv preprint*, 2024, submitted May 12 2024; open-source code provided.
- [17] Z. Ravichandran, F. Cladera, J. Hughes, V. Murali, M. A. Hsieh, G. J. Pappas, C. J. Taylor, and V. Kumar, “Deploying foundation model-enabled air and ground robots in the field: Challenges and opportunities,” *ICRA Workshop on Field Robotics*, 2025.
- [18] M. Fadhil Ginting, K. Otsu, M. J. Kochenderfer, and A.-a. Agha-mohammadi, “Capability-aware task allocation and team formation analysis for cooperative exploration of complex environments,” *International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [19] M. Tranzatto, T. Miki, M. Dharmadhikari, L. Bernreiter, M. Kulkarni, F. Mascari, O. Andersson, S. Khattak, M. Hutter, R. Siegwart *et al.*, “Cerberus in the darpa subterranean challenge,” *Science Robotics*, vol. 7, no. 66, p. eabp9742, 2022.
- [20] F. Cladera, Z. Ravichandran, I. D. Miller, M. A. Hsieh, C. J. Taylor, and V. Kumar, “Enabling large-scale heterogeneous collaboration with opportunistic communications,” in *Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 2610–2616. [Online]. Available: <https://arxiv.org/abs/2309.15975>
- [21] Z. Mandi, S. Jain, and S. Song, “Roco: Dialectic multi-robot collaboration with large language models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299.
- [22] P. Gupta, D. Isele, E. Sachdeva, P.-H. Huang, B. Dariush, K. Lee, and S. Bae, “Generalized mission planning for heterogeneous multi-robot teams via llm-constructed hierarchical trees,” *arXiv preprint arXiv:2501.16539*, 2025.
- [23] J. Strader, A. Ray, J. Arkin, M. B. Peterson, Y. Chang, N. Hughes, C. Bradley, Y. X. Jia, C. Nieto-Granda, R. Talak, C. Fan, L. Carlone, J. P. How, and N. Roy, “Language-grounded hierarchical planning and execution with multi-robot 3d scene graphs,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.07454>
- [24] F. Cladera, Z. Ravichandran, J. Hughes, V. Murali, C. Nieto-Granda, M. Ani Hsieh, G. J. Pappas, C. J. Taylor, and V. Kumar, “Air-ground collaboration for language-specified missions in unknown environments,” *IEEE Transactions on Field Robotics*, pp. 1–1, 2025.
- [25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, 2022.
- [26] D. Giannakopoulou, C. S. Pășăreanu, and J. M. Cobleigh, “Assume-guarantee verification of source code with design-level assumptions,” in *Proceedings of the 26th International Conference on Software Engineering*. IEEE Computer Society, 2004, pp. 211–220.
- [27] Y. Tian, Y. Chang, F. H. Arias, C. Nieto-Granda, J. P. How, and L. Carlone, “Kimera-multi: Robust, distributed, dense metric-semantic slam for multi-robot systems,” *Transactions on Robotics*, 2022.
- [28] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu *et al.*, “Grounding dino: Marrying dino with grounded pre-training for open-set object detection,” *arXiv preprint arXiv:2303.05499*, 2023.
- [29] M. Cai, H. Liu, S. K. Mustikovela, G. P. Meyer, Y. Chai, D. Park, and Y. J. Lee, “Making large multimodal models understand arbitrary visual prompts,” *arXiv:2312.00784*, 2023.
- [30] Z. Ravichandran, I. Hounie, F. Cladera, A. Ribeiro, G. J. Pappas, and V. Kumar, “Distilling on-device language models for robot planning with minimal human intervention,” *Conference on Robot Learning (CoRL)*, 2025.

APPENDIX I SUMMARY

This appendix provides additional implementation and experimental details for SPINE-HT. Section A2 provides implementation details on the LLM prompts (§A2-A), robot behaviors (§A2-B), and UAV mapper (§A2-D) used by our framework. Section A3 provides further experimental details. Section A3-A provides the full list of mission specifications and team configurations used in our experiments. Section A3-B provides further details about our evaluation environments. Sections A3-C and A3-D walk through two experimental missions—one simulated and one real. Finally, Section A4 provides further experimental analysis.

APPENDIX II IMPLEMENTATION DETAILS

A. LLM Configuration

The LLM’s system prompt consists of four main components: a role overview, a definition of available behaviors, the mapping API, and the planning API. The role overview defines the LLM’s high-level objectives (Listing A4-B). The LLM is instructed to generate a plan using a predefined list of behaviors (Listing A4-B), which are described in Section A2-B. The LLM receives semantic map updates via the API defined in Listing A4-B. The final portion of the LLM prompt defines the plan generation format (Listing A4-B). As stated in Section III-C, the LLM begins the mission with input specifications from the user. All updates are provided in-context via the mapping API.

B. Behaviors

We consider five ground robot behaviors and two aerial robot behaviors, which are described in Listing A4-B. These behaviors take grounding arguments comprising entities in the semantic map or queries, which are used to extract detailed and mission-relevant information from a VLM. The ground robot capability arguments (`ugv_type`) allow the planner to specify any UGV, a platform type, or a particular robot (`robot_name`). For example, if the mission requires a robot to traverse rugged terrain, the planner may invoke a navigation behavior for a Husky. The preconditions for behaviors requiring navigation—`ugv_navigate`, `ugv_inspect`, `ugv_map_region`, and `uav_map`—require that a valid path exists in the current semantic map. Because we only consider one UAV in our team, the capability argument of the UAV behaviors is implied. The `ugv_inspect` behavior further requires that the target object is connected to a region node. All behaviors return a boolean value that indicates whether the behavior is successful. Behaviors may also provide information acquired during execution, such as semantic map updates via the interface defined in Listing A4-B.

C. UGV Mapping

As discussed in §IV-A, the UGV semantic mapping framework takes three sources of input: odometry, a Lidar pointcloud, and RGB-D image. We use Direct Lidar Inertial

Odometry for localization [2]. Pointclouds are segmented for obstacles by GroundGrid [13], and these obstacles are used to determine freespace for region and edge placement. GroundingDino performs open-vocabulary object detection [9]. Detections are localized in 3D space, associated via a multiply-hypothesis tracker, and associated tracks are added to the semantic map [12]. Finally, we use the VIP-Llava VLM for region and object captioning [1].

D. UAV Mapping

The UAV provides a sparse map in a graphical structure as shown in Fig. 3 and Fig. A1. The camera module runs its own state estimate system to get accurate metric positions and orientation of the camera. We leverage GTSAM [4] for this system and GPS as prior factors and preintegrate the IMU measurements. We optimize using ISAM2 [7] every time a GPS factor is added. The IMU preintegration allows us to get state estimates at the IMU rate, which is 400hz, allowing us to get an accurate global position for each image. To do the semantic segmentation we use Grounded-SAM2 [11]. The semantic classes are defined by the SPINE-HT planner and sent to the UAV. We predefine the semantic class of “road” as the defining class to place region nodes. When roads are detected and segmented we take the largest road segment and choose the point on the mask that is furthest from other classes, to ensure traversability by the ground robots. New region nodes are added to the closest other road node if it is sufficiently close, otherwise it is only connected to the previous region node. When objects are detected and masked we take the point on the mask this is closest to the road mask and then connect it to the closest region node. The graph is generated incrementally as more images are processed. All the nodes are georegistered since the camera’s position is estimated in the ENU frame. The georegistered graph can then be sent to the ground robots.

To construct this map but first getting task relevant semantic classes from the SPINE-HT planner.

APPENDIX III EXPERIMENTAL DETAILS

A. Mission Specifications

We provide the set of missions and team configurations used in our experiments. Simulated experiments are detailed in Table A1 and real-world missions are shown in Table A4-B. To introduce heterogeneity, some missions designate specific capabilities. For example, during missions that emulate chemical spill triage, one robot was specified to be carrying a chemical containment package. The planner must use this information to appropriately designate subtasks.

B. Experimental Environments

This section provides additional details about experimental environments, including semantics and prior maps used to initialize the semantic mapping framework. We provide a top-down view of the environment used for real-world experiments in Fig. A1. The environment features a range

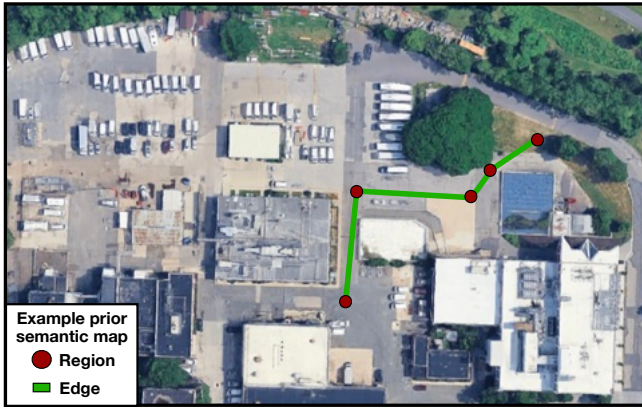


Fig. A1. Top-down view of environment used for real-world experiments with an example prior map overlaid.

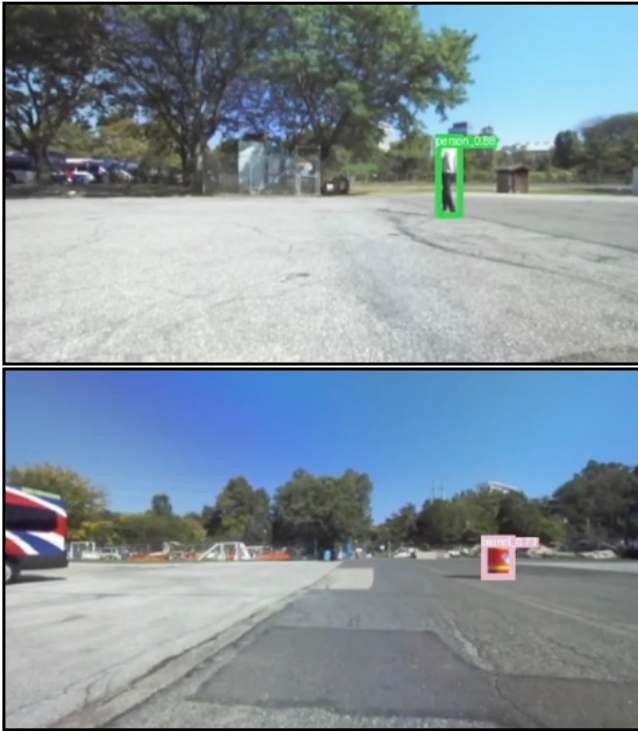


Fig. A2. Semantics discovered by robots and added to their semantic map during the mission described in §A3-C.

of semantics (see Fig. A2) and terrain types (see Fig. A3), which enables complex missions.

C. Real-World Multi-Robot Triage Mission.

We describe a mission where SPINE-HT must triage a potential chemical spill using four robots: a Boston Dynamics Spot, a Clearpath Jackal, a Clearpath Husky, and a high-altitude UAV. All robots start at the same location, as shown in Fig. A4. SPINE-HT was given the following mission specification: “I got reports of a chemical barrel near people around (45, 100) and terrain 1. Establish comms in a central northern location, map the area, and report findings. Use the Spot to map UAV-discovered regions.” The team specification



Fig. A3. Views of experimental environments.

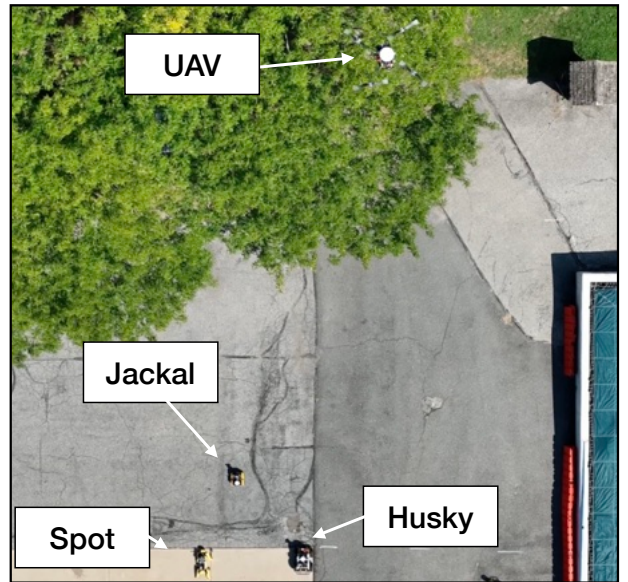


Fig. A4. Robots at the beginning of the experiment described in Section A3-C.

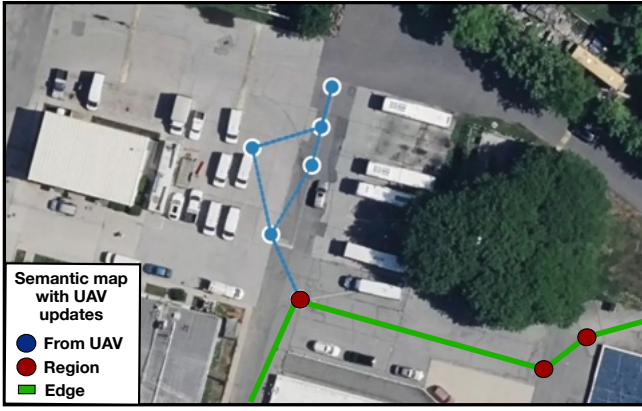


Fig. A5. Semantic map comprising priors and overhead maps constructed by the UAV during the mission described in §A3-C.

was: “You have one Boston Dynamics Spot, one Clearpath Husky, one Clearpath Jackal, and one high-altitude UAV. The Jackal is equipped with a strong radio.” The planner was also given the prior map illustrated in Fig. A1 via JSON (condensed for brevity):

```
"regions": [
  {"name": "ground_1", "coords": [0, 0]},
  {"name": "ground_2", "coords": [0, 20]},
  {"name": "ground_3", "coords": [0, 45]},
  {"name": "terrain_1", "coords": [20, -20],
    "description": "in rough terrain"},
  {"name": "terrain_2", "coords": [6, -35]},
],
"objects": [],
"region_connections": [
  ["ground_1", "terrain_1"],
  ["terrain_1", "terrain_2"],
  ["ground_1", "ground_2"],
  ["ground_2", "ground_3"],
],
"object_connections": []
}
```

This provides a skeleton of the environment used to initialize planning. SPINE-HT tasks the Jackal to establish communication and Husky to explore the rougher terrain, while concurrently tasking the UAV to map more distant areas:

```
uav_explore_to(x=45.0, y=100.0)
ugv_navigate(region_node='ground_3',
  ↳ ugv_type='jackal')
ugv_map_region(region_node='terrain_1',
  ↳ ugv_type='husky')
```

During subtask execution, the Jackal discovered a person (illustrated in Fig. A2, top), the Husky successfully navigated to its goal, and the UAV built an overhead map of the mission-relevant area (Fig. A5). These updates were provided to SPINE-HT via the following in-context update:

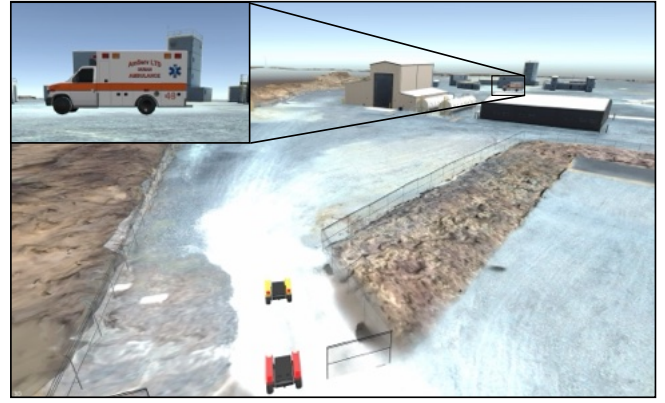


Fig. A6. Simulation environment used for the experiment described in §A3-D. The planner had to deliver a package had to be delivered to the ambulance, whose location was initially unknown.

```
jackal_1 updates:
  add_nodes({name: discovered_person_0, type:
    ↳ object,
              coords: [-3.2, 24.2]}),
  add_connections([discovered_person_0,
    ↳ ground_2]),
  update_robot_location(ground_3)

husky_1 updates:
  update_robot_location(terrain_1)

uav_1 updates:
  add_nodes(
    {coords: [35.4, 54.4], description:
      ↳ from_uav,
      type: region, name: road_1_from_uav},
    {coords: [27.5, 49.9], description:
      ↳ from_uav,
      type: region, name: road_2_from_uav},
    ...
  ),
  add_connections(
    [road_1_from_uav, road_2_from_uav],
    [road_1_from_uav, road_2_from_uav, ...])
```

SPINE-HT then tasked the Spot to explore the first UAV-discovered region and inspect the discovered person for signs of harm via the subtasks:

```
explore_to_node(region_node='road_2_from_uav',
  ugv_type='spot')
ugv_inspect(object_node='discovered_person_0',
  query='signs of chemical exposure
  ↳ or distress',
  ugv_type='spot')
```

During the first subtask, the Spot discovered a chemical barrel (see Fig. A2, bottom). Because the mission explicitly states to discover a chemical barrel, SPINE-HT refined Spot’s subtask sequence to prioritize barrel inspection, which Spot completed successfully and provided the mission-relevant information to the user.

D. Simulated Multi-Robot Care Package Delivery Mission.

This mission requires SPINE-HT to deliver a care package to an ambulance whose location is unknown. SPINE-HT

has access to two Clearpath Warthog robots—Warty and Wanda. Our simulator only provides Clearpath Warthog models, so we introduce heterogeneity by providing individual robots with unique traits. In this mission, Warty is carrying a heavy care package that needs to be delivered to the ambulance once its found. SPINE-HT was given the mission specification: “An ambulance has been reported at one of the building sites. Find it and deliver the package.”, and the team specification “You have two clearpath warthog robots named warty and wanda. warty is carrying a heavy care package and hence moves slower than wanda.” The planner was given a prior map of the environment shown in Fig. A6 as follows (condensed for brevity):

```
"regions": [
  {"name": "region_1", "coords": [0, 0]},
  {"name": "region_2", "coords": [40, -20]},
  {"name": "region_3", "coords": [70, -27]},
  {"name": "region_4", "coords": [53, -101]},
  {"name": "region_5", "coords": [113, -118]}
],
"objects": [
  {"name": "building_1", "coords": [91, -40]},
  {"name": "building_2", "coords": [134, -127]}
],
"region_connections": [
  ["region_1", "region_2"],
  ["region_2", "region_3"],
  ["region_2", "region_4"],
  ["region_4", "region_5"]
],
"object_connections": [
  ["region_3", "building_1"],
  ["region_5", "building_2"]
]
```

SPINE-HT tasks the slower robot, Warty, to inspect the area around the closer building (building_1 and the faster robot, Wanda, to inspect the farther building (building_2):

```
ugv_map_region(region_node='region_3',
↪ ugv_type='warty')
ugv_map_region(region_node='region_5',
↪ ugv_type='wanda')
```

The ambulance is parked near building_2. Wanda finds it, adds it to the semantic map, and notifies SPINE-HT via an in-context update:

```
Wanda updates:
add_nodes({name: discovered_ambulance_0, type:
↪ object,
        coords: [123, -121]}),
add_connections([discovered_ambulance_0,
↪ region_5]),
update_robot_location(region_5)

Warty updates:
update_robot_location(region_3)
```

Having discovered the ambulance, SPINE-HT tasks Warty—the robot carrying the package—to the ambulance.

APPENDIX IV FURTHER EXPERIMENTAL ANALYSIS

This section provides further experimental analysis and discusses promising future directions.

A. Comparative Performance.

As described in Section IV-B, we run the baseline planner [8] on the same set of simulated missions as SPINE-HT, and we observe three main limitations with the baseline planner. Firstly, the baseline planner generates sequential instead of parallelized tasking of robots. For example, at each planning step, the baseline planner will assign only one robot to traverse a single region node in the semantic map, even though it has access to the entire graph and can task robots in parallel directly to the goal location. In contrast, SPINE-HT parallelizes tasking parallelization, which leads to more efficient mission execution as shown in Tab. II. Secondly, the baseline planner struggles to understand the appropriate termination conditions when given a mission with incomplete specifications. For example, the *establish communication* missions listed in Tab. A1 require a robot establish communication by transporting a radio to a particular location. Although the baseline planner provides an initially coherent plan, it does not recognize when it has met the termination conditions of guiding a radio-equipped robot to the goal location. SPINE-HT, on the other hand, understands that sending the robot to a goal location establishes communications. Lastly, the baseline planner often doesn’t understand the mission-relevant implications of certain team specifications. For example, in the care package mission described in §A3-D, the baseline planner infers that having a heavy care package means the robot should not move at all until the ambulance is found. The optimal task allocation, however is to assign the heavier, slower-moving robot to the nearer location and the lighter, faster-moving robot to the farther location, which SPINE-HT accomplishes.

B. Towards a Tool Use Paradigm.

The ability to translate high-level semantic goals into grounded robot plans is a core strength of our framework, and this ability is enabled by LLM reasoning. We observe, however, the LLMs may struggle to reason about lower-level information such as spatially-optimal task assignments, as demonstrated in our ablation studies §IV-D. For example, in one of the simulated missions, the robots were tasked with communicating with each other, but they had to be *within a 30-meter radius* of each other to do so. In situations like these, the planner ignores this spatial constraint in the mission specification and still facilitates communication when robots are beyond range. We envision that effective instantiations of our framework will adhere to a “tool use” paradigm wherein an LLM calls high-level behaviors. For example, several of our experimental missions require a robot to act as a communication relay. Instead of having SPINE-HT directly place robots, future work may leverage existing functionality for efficient communication [3], [5], [6] or exploration [10].

```

# Role and Objective
- Serve as a multi-robot task allocator, generating and refining task allocation plans for a fleet of
↳ robots. Respond dynamically to changes in team composition, mission objectives, and the semantic
↳ graph environment.

# Instructions
- Begin each planning iteration with a concise conceptual checklist (3-7 bullets)
  - Outlining key planning steps (avoid implementation specifics).
  - Identifying critical enabling tasks (e.g., network setup, access clearing) and ensure they
    ↳ precede dependent tasks.
- For each step, input includes: <team specification>, <mission specification>, and <semantic graph>.
- Explicitly identify and list all mission-relevant regions and objects by aligning mission terms to
↳ nodes in the semantic graph.
  - Always attempt to ground mission concepts to existing nodes before generating tasks. Start with
    ↳ the most relevant nodes.
  - If multiple candidate nodes exist, explain the mapping choice.
  - If no relevant nodes exist, state this clearly and fall back to exploration or prerequisite
    ↳ mapping tasks.
- Produce a <team allocation> as a single JSON object, conforming strictly to the output schema
↳ below. Ensure all field specifications and planning constraints are followed exactly.
- Planning is iterative: after each plan execution, you may receive updated feedback or semantic
↳ graph modifications. Revise and regenerate your plan in each new iteration.

## Robot team specification
- {team_specification}
- Available robot APIs are detailed below; only specify robot type in task calls if required by
↳ mission requirements.
- Pay attention to environment semantics and robot capabilities when tasking heterogenous robots.
↳ Some robots may be better at traversing difficult terrain, have more payload capacity, etc.
- Here are the general robot capabilities
  - Husky: Can traverse rugged terrain and is good for finding new paths, however it is slow.
  - Spot: Is fast and good for exploration
  - Jackal: A middleground between the husky and spot

## Robot APIs
- Each robot function is defined below; function signatures and docstrings specify expected
↳ parameters and return fields.
- Assign a specific robot only when the task requires its unique capability. Otherwise, use "any" to
↳ indicate any available robot can perform the task.
When a task explicitly requires a unique capability (e.g., best radio), specify the exact robot ID
↳ (e.g., Jackal_2).
- Try to task all robots, unless dependency conditions are stated in the mission
- For APIs with a robot_name option, specify a particular robot only if there is a good reason (i.e.,
↳ a capability requirement).

```

Listing 1: Role overview

```

def ugv_navigate(
    region_node: str, ugv_type: str = ["any", "jackal", "husky", "robot_name"]
) -> bool:
    """Navigate to a node in the semantic graph. This MUST refer to an existing node.

    Parameters
    ---
    region_node: str
        Existing node in the graph
    ugv_type: str
        Pick most appropriate option (jackal, husky, or any)
    """

def ugv_inspect(
    object_node: str, query: str, ugv_type=["any", "jackal", "spot", "robot_name"]
) -> str:
    """Navigate to the closest region, then inspect `object_node` for a specific attribute `query`.
    The query is processed by a vision language model (VLA), and the VLA's answer will be provided.
    This subsumes navigating to the region closest to the object.
    """

def ugv_map_region(
    region_node: str, ugv_type: str = ["any", "jackal", "husky", "spot", "robot_name"]
) -> str:
    """Navigate to `region_node`, gather a semantic description, and discover nearby objects.
    This subsumes navigation to region_node"""

def ugv_explore_to_coord(
    x: float, y: float, ugv_type: str = ["any", "jackal", "husky", "spot", "robot_name"]
) -> str:
    """Try to navigate to the coordinate (x, y).
    If successful, you will update your semantic map with a node at this location.
    Note that you may not reach the coordinate exactly. Allow for about 10 meters of error.

    Use this if the mission requires navigating to a coordinate that is not close (within 10 meters)
    ↪ to an existing node.
    """

def ugv_explore_to_node(region_node: str, ugv_type=["any", "jackal", "husky", "spot", "robot_name"])
    ↪ -> str:
    """Attempt to find a path to `region_node` if none exists.

    ONLY call this if there is no existing path"""

def uav_map(region_node: str) -> bool:
    """Fly to a node in the graph and map that area. Returns map updates"""

def uav_explore_to(x: float, y: float) -> str:
    """Explore an (x, y) coordinate to expand the semantic map. Returns map updates."""

```

Listing 2: Behavior description

```

## Semantic Graph
- Provided as a JSON object with fields: objects, regions, object_connections, region_connections,
↳ and init_location. Example:

{{
"objects": [{"name": "object_1_name", "coords": [x, y]}],
"regions": [{"name": "region_1_name", "coords": [x, y]}],
"object_connections": [{"object_name", "region_name"}],
"region_connections": [{"region_name_a", "region_name_b"}],
"init_location": "starting_region"
}}

- Graph updates are given via API:

def remove_nodes(removed_nodes: List[str]) -> None:
    """Remove `nodes` and associated edges from graph."""

def add_nodes(new_nodes: List[Dict[str, str]]) -> None:
    """Add nodes to graph. Each node is represented as a dictionary."""

def add_connections(new_connections: List[Tuple[str, str]]) -> None:
    """Add a list of connections. Each element is a tuple of the connecting nodes."""

def remove_connections(removed_connections: List[Tuple[str, str]]) -> None:
    """Removes a list of connections. Each element is a tuple of the endpoint nodes in the edge."""

def update_robot_location(region_node: str) -> None:
    """Update robot's location in the graph to `region_node`."""

def update_node_attributes(attribute: List[Dict[str, str]]) -> None:
    """Update node's attributes. Each entry of the input is a dictionary of new node values.
    Entries will include the referent node's name."""

def no_updates() -> None:
    """There have been no updates."""

```

Listing 3: Mapping API

```

# Planning and Verification
- In each mission cycle:
- Ensure that all proposed tasks are executable within the current semantic graph and robot team.
- Before returning output, verify all tasks for correctness and completeness.

# Make concise plans
- Do not call subsumed tasks. For example, if inspecting an object subsumes navigation, so do not
  ↳ call navigation then inspection unless necessary

# Feasibility and Adapting plans
- Always address infeasibility or feedback by updating your plan, such as generating intermediate
  ↳ subtasks or correcting syntax.
- Do not call tasks until they are feasible. If you receive feedback about a task, it is infeasible.
  ↳ Remove that from your plan and add it during a later iteration.
- Revise in response to feedback or infeasibility by correcting errors or adding intermediate plans;
  ↳ regenerate output.

# Extending plans
- Treat the previous plan as cumulative and authoritative unless instructed to modify otherwise; do
  ↳ not omit or deduplicate executed/planned tasks unless explicitly told to do so.
- Each output iteration should be a superset of the previous plan unless instructed to remove tasks.
- You will be given a list of previously completed tasks. Do not duplicate tasks during successive
  ↳ planning iterations. If the task did not return the designed information, calling that task again
  ↳ will not help.

# Sub-categories
- Use the UAV to explore and map far away locations. Do not use the UAV to map somewhere a UGV can
  ↳ map.
- Ignore malformed inputs; assume all provided data is in the required format.

# Verbosity
- Output a clear, concise JSON matching the specified schema. Include succinct justifications for
  ↳ planning decisions.
- In `tasks`, use explicit parameter names and formatting per the function signatures.
- Preserve field order and types as per the provided specification.

# Stop Conditions
- Terminate planning only upon producing a non-blank `mission_answer` indicating completion.
- In all other cases, extend the plan as a superset of the prior iteration, unless told otherwise.
- Stop once you complete all reasonable tasks and report your findings (even if there are none). Do
  ↳ not repeat the same plan multiple times in a row.

## Output Format
Each response must be a single JSON object with these required fields (names, types, order required):
```json
{{
 "conceptual_checklist": ["steps as outlined above"],
 "robot_team": "summarize robots and their abilities",
 "reasoning": "string: concise rationale for the planning step",
 "relevant_regions": ["most_relevant", "second_most_relevant", ...], // This may include regions you
 ↳ plan to leverage in future planning iterations
 "grounding_explanation": "string: explain how mission terms were mapped to semantic graph nodes, or
 ↳ why fallback was required",
 "tasking_explanation": "Justify why you tasked robots, given their capabilities",
 "relevant_graph": "List relevant portions of the graph, and explain why they are important",
 "mission_answer": "string: completed mission output or blank if still planning",
 "tasks": ["string", ...],
 "dependency_reasoning": "string: explanation of task dependencies",
 "dependency_graph": [{"prior_task", "dependent_task"}],
 "is_extended": true if mission is extended
}}
```

- Include all fields in every output.
- The `tasks` array must consist solely of formatted robot API call strings per the given function
  ↳ signatures.
- The `dependency_graph` array must contain [{"prior_task", "dependent_task"}] pairs exactly matching
  ↳ `tasks` entries. Dependencies must describe mission ordering ONLY for feasible tasks. If a task
  ↳ is infeasible, call it in a later iteration when it becomes feasible.
- If validation errors are found post-output, revise and redo the full JSON object, adhering strictly
  ↳ to this schema.

# Contextual reasoning
- Attend to semantic relationships between the mission specification and graph.
  - Make educated inferences. For example, if asked to find a car, look near the roads. Boats are
    ↳ near docks, etc.

```

Listing 4: Planning instructions

| Mission specifications | Platforms and capability description |
|---|--|
| There are reports of a traffic blockade under the overpass. Report what is going on. | Warthog (x2) |
| There are reports of an emergency situation at the office building. Report what is going on. | Warthog (x2) |
| There is a fire reported at the back the factory building. Report the situation and block the front of the factory for safety. | Warthog (x2) |
| There is a fire reported at the back the factory building. Report the situation and block the front of the factory for safety. | Warthog (x2). Only one of them is equipped with a capable front-end system for object detection and scene understanding. |
| Communications are down between the factory and HQ building. Reestablish comms and report the situation. | Warthog (x2). Both are equipped with radio relays used to establish communication. |
| Important cargo needs to be delivered from the container to the factory. Ensure there are no people or vehicles in the way. | Warthog (x2) |
| There are reports that a tsunami has struck the area. Survey the docks and report any damages to the ships. | Warthog (x2) |
| Check the front gate for signs of an intruder. Also, report the situation at the office. | Warthog (x2). One Warthog has a speaker system to sound an alarm when needed. |
| An ambulance has been reported at one of the building sites. Find it and deliver the package. | Warthog (x2). One Warthog carries a heavy care package and hence moves slower than the other one. |
| There are reports that a tsunami has struck the area. Survey the docks and report any damages to the ships. | Warthog (x3) |
| There is a chemical spill reported at the factory. Investigate the spill, go report the situation to HQ and block the front gate for safety. | Warthog (x3) |
| There is a chemical spill reported at the factory. Investigate the spill, go report the situation to the office and block the front gate for safety. | Warthog (x3). One Warthog is equipped with a containment device to capture any harmful chemicals. |
| There is a chemical spill reported at the factory. Investigate the spill, go report the situation to the office and block the front gate for safety. | Warthog (x3). Warthog 1 is equipped with a containment device to capture any harmful chemicals. Warthog 3 is equipped with a communication radio capable of communicating within a 30 meter radius. |
| Communications are down between the office, HQ and the factory. Help reestablish comms. | Warthog (x3). All three are equipped with radio relays and can communicate with each other. Warthog 3's radio device is of a much higher capacity and can communicate farther than Warthog 1's or Warthog 2's. |
| An intruder has entered from the front gate and was last sighted at the factory. See if that's true, block the front gate for safety, and report the situation at the office. | Warthog (x3). Warthog 1 and Warthog 3 have a speaker system to sound an alarm when needed. |
| There are reports that a tsunami has struck the area. Survey the docks and report any damages to the ships. | Warthog (x4) |

TABLE A1
SIMULATED MISSIONS AND PLATFORM DESCRIPTIONS.

| Specifications | Platforms and capability description |
|---|---|
| Inspect two regions. | Jackal (x2) |
| Obtain the package and watch for oncoming traffic. | Jackal (x2). One jackal has a larger payload capacity. |
| Map the northmost part of the scene. | Jackal (x2) |
| Pickup the package and watch for oncoming cars. | Jackal, Spot. The Spot has a larger payload capacity. |
| Mapping. | Jackal (x2), Spot |
| Inspect key infrastructure. | Husky, Spot. The Spot is faster while the Husky can traverse more rugged terrain. |
| Inspect key infrastructure and establish communications. | Husky, Spot, Jackal. The Spot is the fastest robot in the team, the Husky can traverse rugged terrain, while the Jackal is equipped with a powerful radio. |
| Map the northmost scene. | Husky, Spot, Jackal. The Spot is the fastest robot in the team, the Husky can traverse rugged terrain, while the Jackal is equipped with a powerful radio. |
| Inspect the chemical barrel and deliver containment package. | Husky, Spot. The Spot is the fastest robot on the team, while the Husky carries a containment packages. |
| Find the chemical barrel and deliver the containment package. | Husky, Spot. The Spot is the fastest robot on the team, while the Husky carries a containment packages. |
| Inspect the chemical barrel, deliver the containment package, and block road to oncoming traffic. | Husky, Spot, Jackal. The Spot is the fastest robot on the team, the Husky can traverse rugged terrain, while the Jackal carries a containment packages. |
| Find the chemical barrel, deliver the containment package, and prevent oncoming traffic. | Husky, Spot, Jackal. The Spot is the fastest robot on the team, the Husky can traverse rugged terrain, while the Jackal carries a containment packages. |
| Find the chemical barrel, deliver the containment package, and block road to oncoming traffic. | Husky, Spot, Jackal. The Spot is the fastest robot on the team, the Husky can traverse rugged terrain, while the Jackal carries a containment packages. |
| Establish communications, find the chemical barrel, then deliver the containment package. | Husky, Spot, Jackal. The Spot is the fastest robot on the team, the Husky can traverse rugged terrain, while the Jackal carries a containment packages. |
| I got reports of a chemical spill. Triage the area. | Husky, Spot, Jackal, UAV. The Spot is the fastest robot on the team, the Husky can traverse rugged terrain, the Jackal has moderate speed and traversability capabilities but is equipped with a semantic mapping framework, while the UAV can construct semantic maps from overhead imagery. |
| I got reports of a missing person. Triage the area. | Husky, Spot, Jackal, UAV. The Spot is the fastest robot on the team, the Husky can traverse rugged terrain, the Jackal has moderate speed and traversability capabilities but is equipped with a semantic mapping framework, while the UAV can construct semantic maps from overhead imagery. |

TABLE A2
REAL ROBOT MISSIONS AND PLATFORM DESCRIPTIONS.

REFERENCES

- [1] Mu Cai, Haotian Liu, Siva Karthik Mustikovela, Gregory P. Meyer, Yuning Chai, Dennis Park, and Yong Jae Lee. Making large multi-modal models understand arbitrary visual prompts. *arXiv:2312.00784*, 2023.
- [2] Kenny Chen, Ryan Nemiroff, and Brett T Lopez. Direct lidar-inertial odometry: Lightweight lio with continuous-time motion correction. *International Conference on Robotics and Automation (ICRA)*, 2023.
- [3] Fernando Cladera, Zachary Ravichandran, Ian D. Miller, M. Ani Hsieh, C. J. Taylor, and Vijay Kumar. Enabling large-scale heterogeneous collaboration with opportunistic communications. In *Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2610–2616, 2024.
- [4] Frank Dellaert and GTSAM Contributors. borglab/gtsam, May 2022.
- [5] Jennifer Gielis, Ajay Shankar, and Amanda Prorok. A critical review of communications in multi-robot systems. *Current robotics reports*, 3(4):213–225, 2022.
- [6] Stephanie Gil, Swarun Kumar, Dina Katabi, and Daniela Rus. Adaptive communication in multi-robot systems using directionality of signal strength. *The International Journal of Robotics Research*, 34(7):946–968, 2015.
- [7] Michael Kaess, Hordur Johannsson, Richard Roberts, Viorela Ila, John Leonard, and Frank Dellaert. isam2: Incremental smoothing and mapping with fluid relinearization and incremental variable reordering. In *2011 IEEE International Conference on Robotics and Automation*, pages 3281–3288, 2011.
- [8] Kehui Liu, Zixin Tang, Dong Wang, Zhigang Wang, Xuelong Li, and Bin Zhao. Coherent: Collaboration of heterogeneous multi-robot system with large language models. *ICRA*, 2025.
- [9] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.
- [10] Dexter Ong, Yuezhan Tao, Varun Murali, Igor Spasojevic, Vijay Kumar, and Pratik Chaudhari. Atlas navigator: Active task-driven language-embedded gaussian splatting. *arXiv:2502.20386*, 2025.
- [11] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Roland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos, 2024.
- [12] Donald B Reid. An algorithm for tracking multiple targets. *IEEE Transactions on Automatic Control*, 24(6):843–854, 1979.
- [13] Nicolai Steinke, Daniel Goehring, and Raúl Rojas. Groundgrid: Lidar point cloud ground segmentation and terrain estimation. *IEEE Robotics and Automation Letters*, 9(1):420–426, 2024.